



BAHIR DAR UNIVERSITY

Bahir Dar Institute of Technology

Faculty of Computing

Department of Information Technology

Operating System

Group Assignment

Group Members	ID
1.Wubalem Wolelaw.....	BDU1702758
2.Azeb Getachew.....	BDU1701624
3.Fana Enyew.....	BDU1701953
4.Maranata Abebe.....	BDU1702288
5.Ribka Fikrie.....	BDU1702526
6.Abel Tilahun.....	BDU1701451
7.Mahlet Aysheshim.....	BDU1604518
8.Abrham Nigussie.....	BDU1701487
9.Selemon Kidane.....	BDU1702592
10.Yeabtsega Bizualem.....	BDU1702781
11.Seida Esmael.....	BDU1604796

Submitted to:Samuel A.(MSc)

CPU Scheduling

When a computer is multi programmed, it frequently has multiple processes competing for the CPU at the same time and when these processes are there in the ready state than the number of available CPUs, the operating system must decide which process to run first.

CPU Scheduling is deciding which process to execute and when to execute each process in order to use the efficient use of the CPU. The Algorithm we use to perform each scheduling is called Scheduling algorithm. Scheduling algorithms can be divided into two categories with respect to how they deal with clock interrupts.

Preemptive scheduling: allows releasing the current executing process from CPU when another process comes and need execution.

Non-preemptive scheduling: once the CPU has been allocated to a process, the process keeps the CPU until it release the CPU .

We use different terms to express time periods each process used while being processed and while waiting for to be executed such as,

Turnaround time:For a particular process, it is the total time needed for process execution It is the sum of process execution time and its waiting times.

Waiting time:The waiting time for a specific process is the sum of all periods it spends waiting in the ready queue.

Response time:It is the time from the submission of a process until the first response is produced (the time the process takes to start responding).

Scheduling Algorithms

1.First Come First Serviced (FCFS) algorithm

The process that comes first will be executed first. it is Not preemptive(The first job is allowed to run as long as it wants to be executed.).It is easy to understand and equally easy to program. In this algorithm processes are assigned the CPU in the order they request it. Basically, there is a single queue of ready processes. When the first job enters the system from the outside , it is started immediately and allowed to run as long as it wants to. As other jobs come in, they are put onto the end of the queue. When the running process blocks, the first process on the queue is run next.

2.Shortest-Job-First (SJF) scheduling

When CPU is available, it will be assigned to the process with the smallest CPU burst (non preemptive). If two processes have the same next CPU burst, FCFS is used. When several equally important jobs are sitting in the input queue waiting to be started, the scheduler picks the shortest job first. Shortest job first is provably optimal when all the jobs are available simultaneously.

3.Shortest-Remaining-Time-First (SRTF)

It is a preemptive version of the Shortest Job First Algorithm. It allows a new process to gain the processor if its execution time is less than the remaining time of the currently processing one. When a new job arrives, its total time is compared to the current process' remaining time. If the new job needs less time to finish than the current process, the current process is suspended and the new job started.

4.Round Robin scheduling

It is one of the oldest and most widely used algorithms. It allocates the CPU for one Quantum time Q to each process in the ready queue. If the process has blocked or finished before the quantum has elapsed, the CPU switching is done when the process block. This scheme is repeated until all processes are finished.

5. Banker's Algorithm

A deadlock happens when processes are permanently stuck waiting for resources that are held by each other. Because all the processes are waiting, none of them will ever cause any event that could wake up any of the other members of the set, and all the processes continue to wait forever.

In most cases, the event that each process is waiting for is the release of some resource currently possessed by another member of the set. In other words, each member of the set of deadlocked processes is waiting for a resource that is owned by a deadlocked process. None of the processes can run and none of them can release any resources.

The main algorithms for deadlock avoidance are based on the concept of safe states. A state is said to be safe if there is some scheduling order in which every process can run to completion even if all of them suddenly request their maximum number of resources immediately.

The Banker's Algorithm acts like a cautious bank manager to avoid this. Before the Operating System grants a resource request, it simulates the allocation. It checks if doing so leaves the system in a "safe state", meaning there is at least one sequence of execution where all processes can finish without deadlocking. If a safe sequence exists, the request is granted; if not, the process must wait.

THE CODE

```
#include <iostream>

#include <vector>

#include <iomanip>

#include <string>

#include <algorithm>

#include <queue>

using namespace std;

// Structure to hold all data for a single process

struct Process {

    string id;

    int arrival_time;

    int burst_time;

    int remaining_time;

    int completion_time;

    int turnaround_time;

    int waiting_time;

};

//Vector to store our processes

vector<Process> processes;
```

```
//Function Prototypes
```

```
void input_processes() {
```

```
    int n;
```

```
    cout << "Enter number of processes: ";
```

```
    cin >> n;
```

```
    for (int i = 0; i < n; i++) {
```

```
        Process p;
```

```
        cout << "Enter Process ID, Arrival Time, and Burst Time: ";
```

```
        cin >> p.id >> p.arrival_time >> p.burst_time;
```

```
        p.remaining_time = p.burst_time;
```

```
        processes.push_back(p);
```

```
    }
```

```
}
```

```
void fcfs(vector<Process> p) {
```

```
    int current_time = 0;
```

```
    float total_wt = 0, total_tat = 0;
```

```
    for (int i = 0; i < p.size(); i++) {
```

```
        if (current_time < p[i].arrival_time) {
```

```
            current_time = p[i].arrival_time;
```

```
        }
```

```
        p[i].waiting_time = current_time - p[i].arrival_time;
```

```
        p[i].completion_time = current_time + p[i].burst_time;
```

```
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
```

```

    total_wt += p[i].waiting_time;

    total_tat += p[i].turnaround_time;

    current_time = p[i].completion_time;
}

// Print Average WT and TAT

cout << "\nID\tAT\tBT\tCT\tTAT\tWT\n";

for (auto &proc : p) {

    cout << proc.id << "\t"

        << proc.arrival_time << "\t"

        << proc.burst_time << "\t"

        << proc.completion_time << "\t"

        << proc.turnaround_time << "\t"

        << proc.waiting_time << endl;

}

cout << "\nAverage Waiting Time = " << total_wt / p.size();

cout << "\nAverage Turnaround Time = " << total_tat / p.size() << endl;}

void sjf_non_preemptive(vector<Process> &p) {

    int current_time = 0, completed = 0;

    int n = p.size();

    vector<bool> is_completed(n, false);

    while (completed != n) {

        int idx = -1;

```

```
int min_burst = 1e9;
```

```
for (int i = 0; i < n; i++) {
```

```
    if (p[i].arrival_time <= current_time && !is_completed[i]) {
```

```
        if (p[i].burst_time < min_burst) {
```

```
            min_burst = p[i].burst_time;
```

```
            idx = i;
```

```
        }
```

```
    }
```

```
}
```

```
if (idx != -1) {
```

```
    p[idx].waiting_time = current_time - p[idx].arrival_time;
```

```
    p[idx].completion_time = current_time + p[idx].burst_time;
```

```
    p[idx].turnaround_time = p[idx].completion_time - p[idx].arrival_time;
```

```
    is_completed[idx] = true;
```

```
    current_time = p[idx].completion_time;
```

```
    completed++;
```

```
} else {
```

```
    current_time++;
```

```
}
```

```
}
```

```
float total_wt = 0, total_tat = 0;
```

```
cout << "\nID\tAT\tBT\tCT\tTAT\tWT\n";
```

```

for (auto &proc : p) {

    total_wt += proc.waiting_time;

    total_tat += proc.turnaround_time;


    cout << proc.id << "\t"

        << proc.arrival_time << "\t"

        << proc.burst_time << "\t"

        << proc.completion_time << "\t"

        << proc.turnaround_time << "\t"

        << proc.waiting_time << endl;

}


cout << "\nAverage Waiting Time = " << total_wt / n;

cout << "\nAverage Turnaround Time = " << total_tat / n << endl;

}

void srtf(vector<Process> p) {

    int current_time = 0, completed = 0, n = p.size();

    int min_rem = 1e9, shortest = 0;

    bool check = false;

    while (completed != n) {

        for (int i = 0; i < n; i++) {

            if ((p[i].arrival_time <= current_time) && (p[i].remaining_time < min_rem) &&
p[i].remaining_time > 0) {

                min_rem = p[i].remaining_time;

```



```
    shortest = i;

    check = true;

}

}
```

```
if (!check) { current_time++; continue; }
```

```
p[shortest].remaining_time--;

min_rem = p[shortest].remaining_time;

if (min_rem == 0) min_rem = 1e9;
```

```
if (p[shortest].remaining_time == 0) {

    completed++;

    check = false;

    p[shortest].completion_time = current_time + 1;

    p[shortest].turnaround_time = p[shortest].completion_time - p[shortest].arrival_time;

    p[shortest].waiting_time = p[shortest].turnaround_time - p[shortest].burst_time;

}

current_time++;

}
```

```
float total_wt = 0, total_tat = 0;
```

```
cout << "\nID\tAT\tBT\tCT\tTAT\tWT\n";
```

```
for (auto &proc : p) {

    total_wt += proc.waiting_time;
```

```
total_tat += proc.turnaround_time;
```

```
cout << proc.id << "\t"
```

```
    << proc.arrival_time << "\t"
```

```
    << proc.burst_time << "\t"
```

```
    << proc.completion_time << "\t"
```

```
    << proc.turnaround_time << "\t"
```

```
    << proc.waiting_time << endl;
```

```
}
```

```
cout << "\nAverage Waiting Time = " << total_wt / n;
```

```
cout << "\nAverage Turnaround Time = " << total_tat / n << endl;}
```

```
void round_robin(vector<Process> p, int quantum) {
```

```
    sort(p.begin(), p.end(), [](Process a, Process b) {
```

```
        return a.arrival_time < b.arrival_time;
```

```
    });
```

```
int current_time = 0, completed = 0, n = p.size();
```

```
queue<int> q;
```

```
vector<bool> in_queue(n, false);
```

```
q.push(0);
```

```
in_queue[0] = true;
```

```
current_time = p[0].arrival_time;
```

```

while (completed != n) {

    if (q.empty()) {

        current_time++;

        for (int i = 0; i < n; i++) {

            if (p[i].arrival_time <= current_time && !in_queue[i] && p[i].remaining_time > 0) {

                q.push(i);

                in_queue[i] = true;

            }

        }

        continue;

    }

}

```

```

int idx = q.front();

```

```

q.pop();

```

```

if (p[idx].remaining_time > quantum) {

```

```

    current_time += quantum;

```

```

    p[idx].remaining_time -= quantum;

```

```

for (int i = 0; i < n; i++) {

```

```

    if (p[i].arrival_time <= current_time && !in_queue[i] && p[i].remaining_time > 0) {

```

```

        q.push(i);

```

```

        in_queue[i] = true;

```

```

    }

```

```

    }

    q.push(idx);

} else {

    current_time += p[idx].remaining_time;

    p[idx].remaining_time = 0;

    completed++;

    p[idx].completion_time = current_time;

    p[idx].turnaround_time = current_time - p[idx].arrival_time;

    p[idx].waiting_time = p[idx].turnaround_time - p[idx].burst_time;

    for (int i = 0; i < n; i++) {

        if (p[i].arrival_time <= current_time && !in_queue[i] && p[i].remaining_time > 0) {

            q.push(i);

            in_queue[i] = true;

        }

    }

}

}

float total_wt = 0, total_tat = 0;

cout << "\nID\tAT\tBT\tCT\tTAT\tWT\n";

```

```

for (int i = 0; i < n; i++) {

    total_wt += p[i].waiting_time;

    total_tat += p[i].turnaround_time;


    cout << p[i].id << "\t"

        << p[i].arrival_time << "\t"

        << p[i].burst_time << "\t"

        << p[i].completion_time << "\t"

        << p[i].turnaround_time << "\t"

        << p[i].waiting_time << "\n";

}


cout << fixed << setprecision(2);

cout << "\nAverage Waiting Time = " << total_wt / n;

cout << "\nAverage Turnaround Time = " << total_tat / n << endl;

}

bool isSafe(vector<int> processes, vector<int> avail,

    vector<vector<int>>> max,

    vector<vector<int>>> allot,

    int P, int R) {

    vector<vector<int>>> need(P, vector<int>(R));

    vector<bool> finish(P, false);

    vector<int> safeSeq(P);

    vector<int> work = avail;

```

```
// Calculate Need

for (int i = 0; i < P; i++)

    for (int j = 0; j < R; j++)

        need[i][j] = max[i][j] - allot[i][j];
```

```
// Print Need matrix

cout << "\nNeed Matrix:\n";

for (int i = 0; i < P; i++) {

    for (int j = 0; j < R; j++) {

        cout << need[i][j] << " ";

    }

    cout << endl;

}
```

```
int count = 0;
```

```
while (count < P) {

    bool found = false;

    for (int p = 0; p < P; p++) {

        if (!finish[p]) {

            int j;

            for (j = 0; j < R; j++)
```

```

        if (need[p][j] > work[j])

            break;

    if (j == R) {

        for (int k = 0; k < R; k++)

            work[k] += allot[p][k];

        safeSeq[count++] = p;

        finish[p] = true;

        found = true;

    }

}

if (!found) {

    cout << "\nSystem is NOT SAFE\n";

    return false;

}

}

cout << "\nSystem is SAFE\nSafe sequence: ";

for (int i = 0; i < P; i++)

    cout << "P" << safeSeq[i] << " ";

cout << endl;

```

```

    return true;
}

int main() {

    //Gather input from the user

    input_processes();

    cout << "\n--- Banker's Algorithm (User Input) ---\n";


    int P, R;

    cout << "Enter number of processes: ";

    cin >> P;


    cout << "Enter number of resources: ";

    cin >> R;


    vector<int> processes_id(P);

    for (int i = 0; i < P; i++) processes_id[i] = i;


    vector<int> avail(R);

    cout << "Enter Available resources:\n";

    for (int i = 0; i < R; i++) cin >> avail[i];


    vector<vector<int>> max(P, vector<int>(R));

    cout << "Enter Max matrix:\n";

    for (int i = 0; i < P; i++)

```



```

    for (int j = 0; j < R; j++)

        cin >> max[i][j];

vector<vector<int>> allot(P, vector<int>(R));

cout << "Enter Allocation matrix:\n";

for (int i = 0; i < P; i++)

    for (int j = 0; j < R; j++)

        cin >> allot[i][j];

cout << "\n--- First Come First Served (FCFS) ---" << endl;

fcfs(processes);

cout << "\n--- Shortest Job First (Non-Preemptive) ---" << endl;

sjf_non_preemptive(processes);

cout << "\n--- Shortest Remaining Time First (SRTF) ---" << endl;

srtf(processes);

cout << "\n--- Round Robin (Time Quantum = 2) ---" << endl;

round_robin(processes, 2);

isSafe(processes_id, avail, max, allot, P, R);

return 0;

}

```

Sample Input and Output

```
Enter number of processes: 3
Enter Process ID, Arrival Time, and Burst Time: P1
0
4
Enter Process ID, Arrival Time, and Burst Time: P2
0
5
Enter Process ID, Arrival Time, and Burst Time: P3
0
3

--- Banker's Algorithm (User Input) ---
Enter number of processes: 5
Enter number of resources: 3
Enter Available resources:
3
3
2
Enter Max matrix:
7
5
3
3
2
2
9
0
2
2
2
2
4
3
```

```

Enter Max matrix:
7
5
3
3
2
2
9
0
2
2
2
2
4
3
3
Enter Allocation matrix:
0
1
0
2
0
0
3
0
2
2
1
1
0
0

```

```

--- First Come First Served (FCFS) ---

ID      AT      BT      CT      TAT      WT
P1       0       4       4       4       0
P2       0       5       9       9       4
P3       0       3      12      12       9

Average Waiting Time = 4.33333
Average Turnaround Time = 8.33333

--- Shortest Job First (Non-Preemptive) ---

ID      AT      BT      CT      TAT      WT
P1       0       4       7       7       3
P2       0       5      12      12       7
P3       0       3       3       3       0

Average Waiting Time = 3.33333
Average Turnaround Time = 7.33333

--- Shortest Remaining Time First (SRTF) ---

ID      AT      BT      CT      TAT      WT
P1       0       4       7       7       3
P2       0       5      12      12       7
P3       0       3       3       3       0

Average Waiting Time = 3.33333
Average Turnaround Time = 7.33333

--- Round Robin (Time Quantum = 2) ---

```

--- Round Robin (Time Quantum = 2) ---

ID	AT	BT	CT	TAT	WT
P1	0	4	8	8	4
P2	0	5	12	12	7
P3	0	3	11	11	8

Average Waiting Time = 6.33

Average Turnaround Time = 10.33

Need Matrix:

7	4	3
1	2	2
6	0	0
0	1	1
4	3	1

System is SAFE

Safe sequence: P1 P3 P4 P0 P2

Process returned 0 (0x0) execution time : 303.630 s

Press any key to continue.